Snip

PROJECT DOCUMENTATION

Distributed URL Shortener with Real-Time Analytics

A polyglot, distributed system demonstrating production-grade architecture patterns: cache-aside redirects, async event-driven analytics over RabbitMQ, cross-service HTTP calls with graceful degradation, and full containerized orchestration.

3	6	72	9,855
Languages	Services	Automated Tests	Load Test Requests, 0 Failures

GitHub Repository: **github.com/mahetab1357/url-shortener**

Author: Mahetab Shaikh

Document generated: June 30, 2026

Table of Contents

Document map

How to read this document

Each section pairs a real diagram or table with a short "why" callout explaining the engineering reasoning behind that decision — written so that both the architecture and the trade-offs reasoning are clear to a reader unfamiliar with the codebase.

1 Executive Summary

What this project is and why it exists

Snip is a full-stack URL shortener built specifically to demonstrate distributed-systems fundamentals rather than just CRUD plumbing: a caching strategy with a real hit/miss story, an asynchronous event pipeline decoupling a latency-critical path from analytics processing, cross-service communication with graceful degradation, and a containerized multi-service deployment with CI verification.

3	6	72	166
Languages — Python, Java, JavaScript	Services in docker-compose	Tests — 49 Python + 23 Java	req/s sustained, 0 failures

What it does

- Shortens any HTTP(S) URL to a collision-safe 7-character code, with optional custom alias and expiry
- Redirects through a Redis cache-aside layer so repeat clicks never hit Postgres
- Publishes every click as an event to RabbitMQ, asynchronously, without slowing the redirect
- Aggregates clicks in a separate Java service: hourly/daily buckets, device/browser breakdown, referrer breakdown
- Serves a live-polling React dashboard with bar and pie charts

> **WHY THIS MATTERS FOR EVALUATION**
>
> This isn't a tutorial clone. Every architectural choice below has a documented trade-off, and several were validated (or corrected) by actually running the system under load — including a real concurrency bug found and fixed during load testing (Section 9).

At a glance

Python · FastAPI | Java · Spring Boot | React · Vite | PostgreSQL | Redis | RabbitMQ | Docker Compose | GitHub Actions CI | Locust Load Testing

2 System Architecture

How the six services fit together

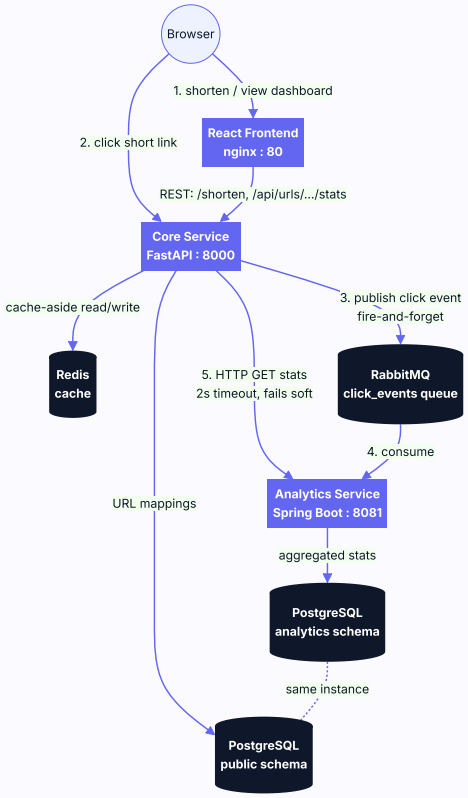

Fig 2.1 — Full system architecture. Numbered arrows show the order of operations for a redirect.

Component responsibilities

Component	Responsibility	Owns
core-service	URL shortening, cached redirects, click-event publishing, cross-service stats aggregation endpoint	Postgres `public` schema
analytics-service	Consumes click events, aggregates hourly/daily/device/referrer stats, serves them over REST	Postgres `analytics` schema
frontend	Shorten UI, live-polling dashboard with charts	Browser-side `localStorage` (which links are "yours")
PostgreSQL	Durable storage, two logically isolated schemas in one instance	—
Redis	Cache-aside layer on the redirect hot path	—

RabbitMQ	Durable, async transport for click events	—

3 Request Flow Diagrams

Step-by-step decision flow for the two core operations

Redirect flow — cache-aside with async publish

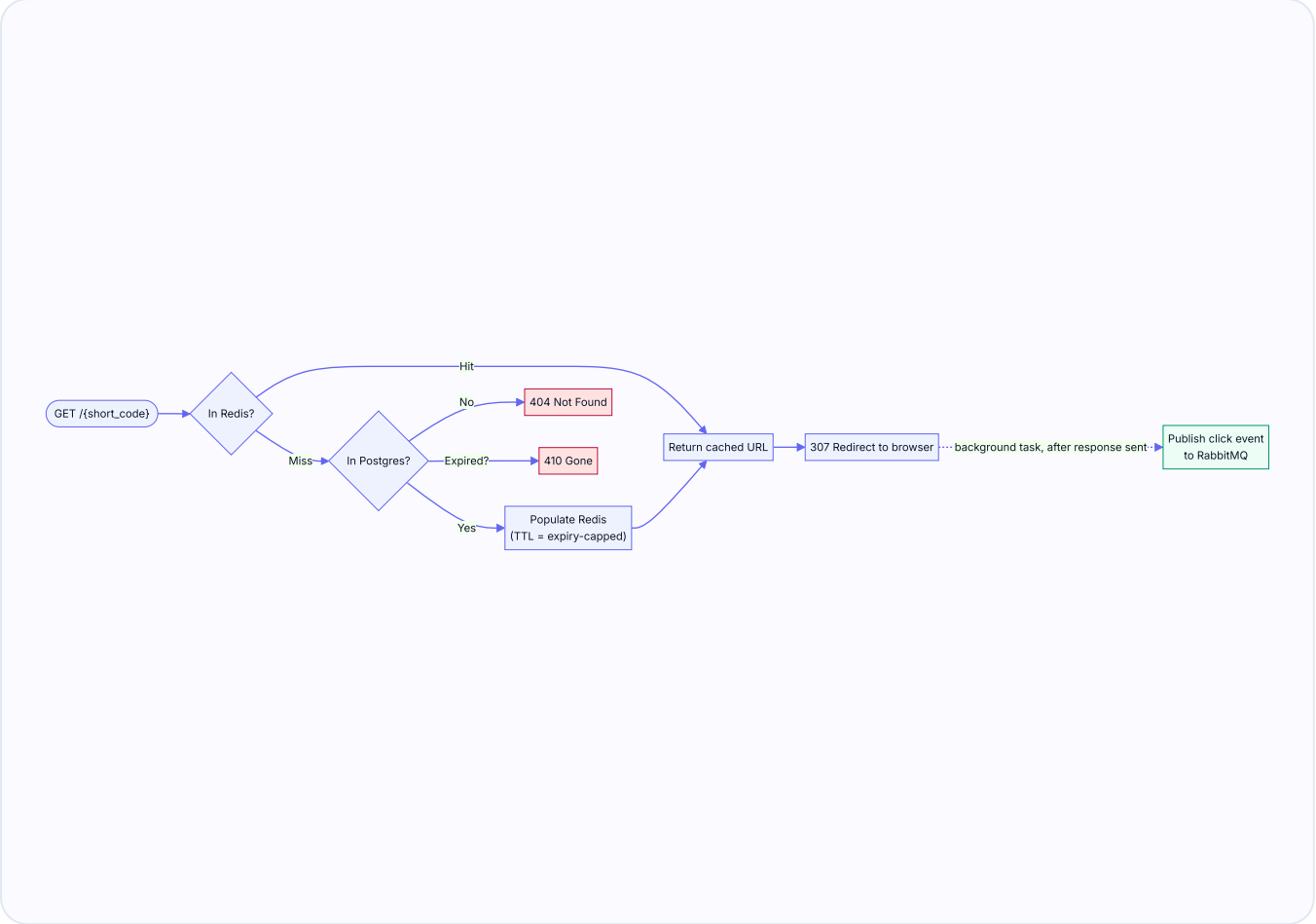

Fig 3.1 — The 307 redirect is sent to the browser before the click event is even published, so queue latency never delays the user.

Stats endpoint — graceful degradation

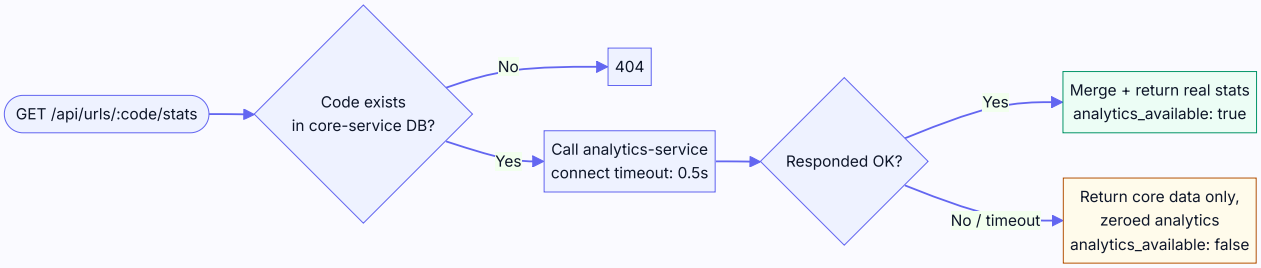

Fig 3.2 — A down analytics-service degrades the response, never fails it. HTTP 200 either way.

WHY GRACEFUL DEGRADATION, NOT A HARD FAILURE

The connect timeout is deliberately tight (0.5s) — a naive single-float timeout was measured taking ~4s on Windows due to sequential IPv6/IPv4 connection attempts, directly undermining the "real-time dashboard" pitch. See Section 4 for the full story.

4 Design Decisions & Rationale

The "why" behind every major architectural choice

Why a separate analytics service?

The redirect path is latency- and availability-critical — every click a user generates goes through it. Aggregation (parsing user agents, bucketing into hourly windows) is real work with no business running synchronously inside a redirect. Splitting it into its own JVM process means a slow or crashing analytics-service can never make redirects slower or fail.

Why Redis caching?

Once a short URL is resolved, the mapping never changes (no edit endpoint exists) — an ideal cache-aside candidate where hits never go stale. Postgres is queried once per code, cold, and never again until the TTL (capped at the link's own expiry) elapses.

Why RabbitMQ over Redis pub/sub?

Redis pub/sub is fire-and-forget: a subscriber that's down when a message publishes loses it permanently. RabbitMQ queues are durable — analytics-service can crash or redeploy and still catch up afterward. The cost (~150-200MB heavier container) is trivial for this project's scale.

Why polling, not WebSockets, for the dashboard?

A WebSocket implementation needs connection-lifecycle management and, if ever scaled past one instance, a pub/sub fan-out layer just to broadcast a number that changes a few times a minute. Five-second polling is a few lines of code for a property that's fine being briefly stale.

Decision	Alternative considered	Why this won
Random short codes + collision retry	ID-encoding (base62 of auto-increment PK)	ID-encoding is guessable/enumerable (id=126 is "the next one"); random codes aren't, at the cost of a (vanishingly rare) collision retry
307 Temporary Redirect	301 Permanent Redirect	301 is cached indefinitely by browsers — future clicks would never reach the server again, breaking click tracking entirely
Read-then-increment counters in analytics-service	Atomic SQL upserts	Correct because a single RabbitMQ consumer thread processes messages sequentially — no concurrent writer exists (today). Documented as a scaling limitation, not hidden.
Hand-rolled user-agent parser	uap-java library	Avoids an external Maven dependency for a feature where rough accuracy is enough at this scale

5 Technology Stack

Language, framework, and purpose for every component

Component	Language / Framework	Purpose
core-service	Python 3.13, FastAPI, SQLAlchemy 2.0, Pydantic v2	URL shortening, cached redirects, event publishing, stats aggregation
analytics-service	Java 21, Spring Boot 3.3, Spring Data JPA, Spring AMQP	Click-event consumption, aggregation, REST stats API
frontend	React 18, Vite, React Router, Recharts	Shorten UI, live analytics dashboard
Database	PostgreSQL 16	Persistent storage, two isolated schemas
Cache	Redis 7	Cache-aside layer for the redirect hot path
Message queue	RabbitMQ 3.13 (management image)	Durable async click-event delivery
Load testing	Locust	Concurrent-request simulation
CI	GitHub Actions	Test, build, optional Docker Hub push
Orchestration	Docker Compose	One-command full-stack startup, dev hot-reload override

Why this language split specifically

The polyglot choice is deliberate, not incidental: Python/FastAPI for a fast-iteration, I/O-bound redirect service; Java/Spring Boot for the aggregation service where the JVM's threading/queue-consumer ecosystem (Spring AMQP) is mature and idiomatic; React for a component-driven dashboard UI. Each language is used where it's the natural fit, not forced.

6 Database Schema

Two schemas, one Postgres instance, zero foreign keys between them

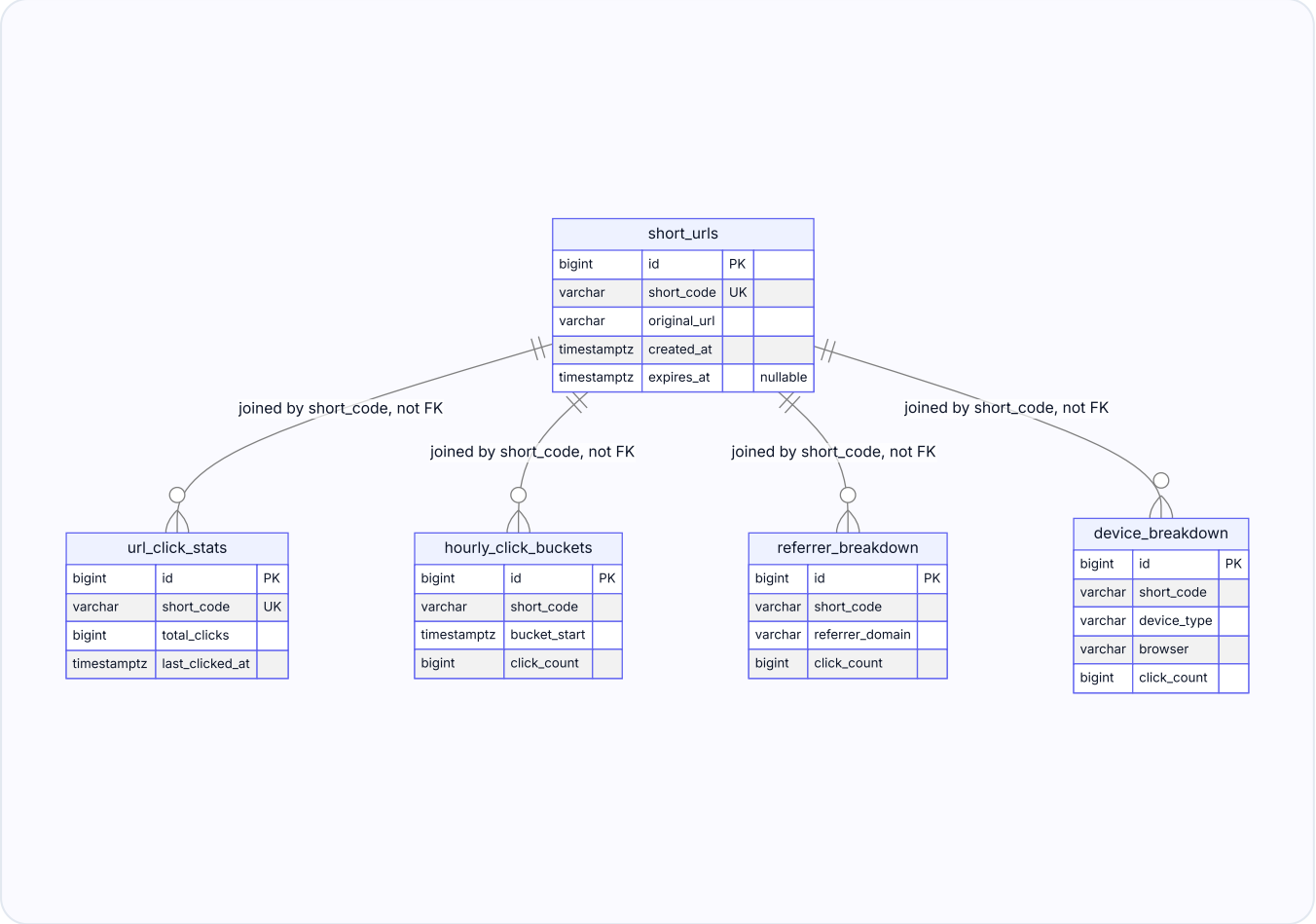

Fig 6.1 — short_urls lives in the public schema (core-service); the four analytics tables live in the analytics schema. No cross-schema foreign keys.

WHY NO FOREIGN KEYS ACROSS SCHEMAS

short_urls is owned exclusively by core-service; the analytics tables exclusively by analytics-service. They're joined only logically, by short_code, and only in application code (an HTTP call) — this is what makes it safe for either service's schema to change independently of the other.

Schema initialization strategy

core-service runs `python -m app.db.init_db` once at container start (not on every app boot — an earlier design coupling schema creation to FastAPI's lifespan hook caused tests to try connecting to a real Postgres). analytics-service creates its schema via a `schema.sql` bootstrap script, then Hibernate's `ddl-auto: update` creates tables inside it. Both are dev-appropriate shortcuts — a production system would use Alembic / Flyway migrations instead.

7 API Reference

Every endpoint, with example requests and status codes

POST /shorten

Create a short URL. Optional: `custom_alias` (3-30 chars), `expires_at` (ISO 8601 with timezone).

Status	Meaning
201	Created
422	Malformed URL or invalid alias format
409	Custom alias already taken
503	Could not allocate a random code after 5 attempts ($62^7 \approx 3.5$ trillion combinations — practically never)

GET /{short_code}

Redirects to the original URL via **307** (not 301 — see Section 4 for why).

Status	Meaning
307	Redirect
404	Unknown short code
410	Link has expired

GET /api/urls/{short_code}/stats

Aggregated stats — core-service's own data merged with a live call to analytics-service. Always returns **200**; degrades to zeroed analytics if analytics-service is unreachable.

GET /api/analytics/{shortCode}/stats

analytics-service's raw endpoint (camelCase JSON — Java/Jackson convention). core-service translates this to snake_case before returning it to the frontend. Returns zeroed stats (not 404) for an unknown code — analytics-service doesn't own short-code validity.

Health checks

Endpoint	Response
`GET /health` (core-service)	`{"status":"ok"}`

`GET /actuator/health` (analytics-service)	`{"status":"UP"}`

8 Testing Strategy & Results

72 automated tests across two languages

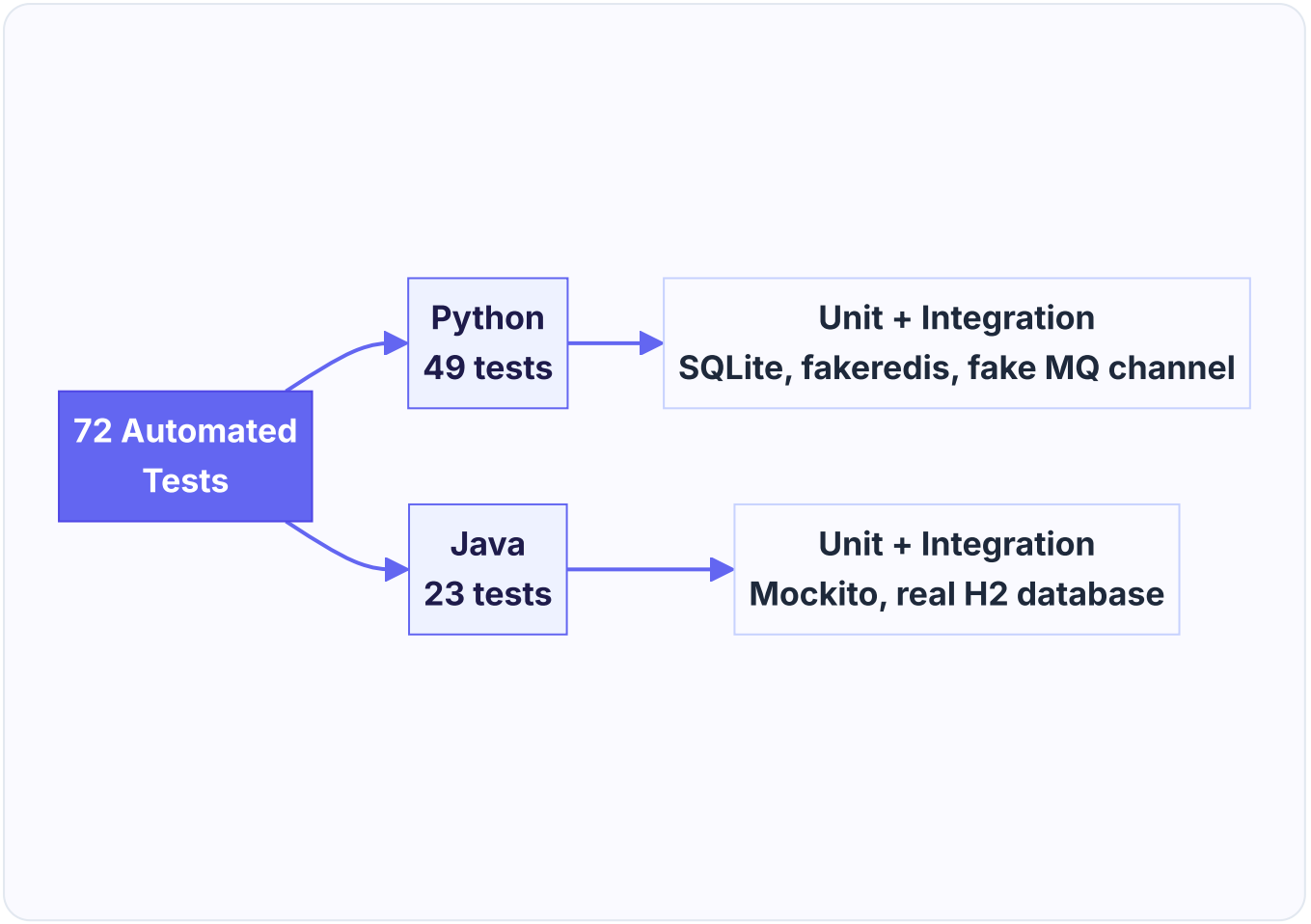

Fig 8.1 — Test suite breakdown by language. Full layer detail in the table below.

Why SQLite/fakeredis instead of real Postgres/Redis for tests

A deliberate tradeoff: millisecond-fast CI feedback over perfect environment parity. Postgres/Redis-specific behavior is exercised by Docker Compose integration testing (Section 11), not pytest. The Java integration test goes further — it runs against a real H2 database (Postgres-compatibility mode) to prove the read-then-increment aggregation logic actually accumulates correctly across multiple events, not just that mocked methods get called.

Suite	Count	What it proves
core-service unit tests	~30	Short-code charset/length/uniqueness, URL validation, collision retry logic
core-service integration tests	~19	Full API behavior: shorten, redirect, cache hit/miss, click publishing, stats degradation
analytics-service unit tests	~20	UA parsing, referrer extraction, aggregation logic via Mockito

analytics-service integration tests	3	Real H2-backed accumulation correctness, REST controller shape

9 Load Testing & the Concurrency Bug

100 concurrent users, 60 seconds, against the real docker-compose stack

A REAL BUG THIS CAUGHT

The first load test run exposed a genuine concurrency bug: core-service's RabbitMQ publisher reused a single `pika.BlockingConnection` across requests. `pika` documents this as not thread-safe — but FastAPI runs sync endpoints across a real thread pool, so concurrent redirects called `publish()` concurrently. Under 100 users,

~8,500 OF ~8,650 PUBLISH ATTEMPTS FAILED SILENTLY

(`ChannelWrongStateError`) — invisible at the HTTP layer, since the redirect path is designed to never fail on a queue problem. Fixed with a `threading.Lock`; verified with a new regression test (20 real threads, proven to fail 3/3 times without the lock) and by re-running the load test and confirming click counts landed correctly in Postgres.

Results — post-fix

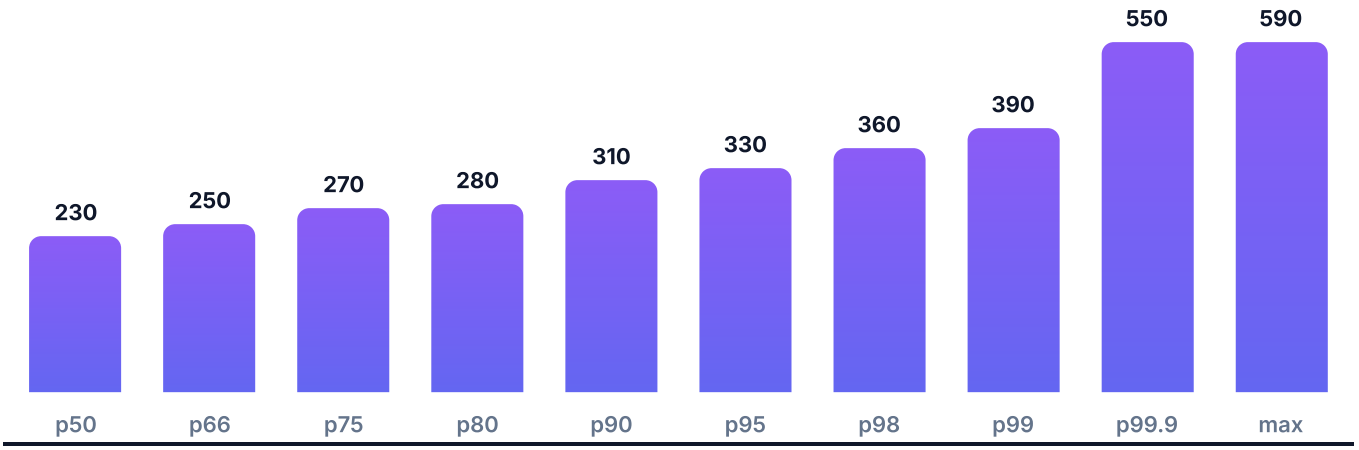

Fig 9.1 — Aggregated response time (ms) by percentile, real Locust output, 9,855 requests.

Endpoint	Requests	Failures	Median	p95	p99	Throughput
GET /{short_code}	9,335	0	230ms	330ms	390ms	157 req/s
POST /shorten	520	0	220ms	320ms	370ms	8.7 req/s
Aggregated	**9,855**	**0 (0.00%)**	**230ms**	**330ms**	**390ms**	**166 req/s**

Tested on a single laptop (AMD Ryzen 5 7530U, 7.4GB RAM) running Locust, Docker Desktop, and all 6 containers simultaneously — not a dedicated load-generator. Latency is dominated by shared-host contention, not application logic. Full results: `load-tests/RESULTS.md` in the repository.

10 CI/CD Pipeline

5-job GitHub Actions workflow, verified green

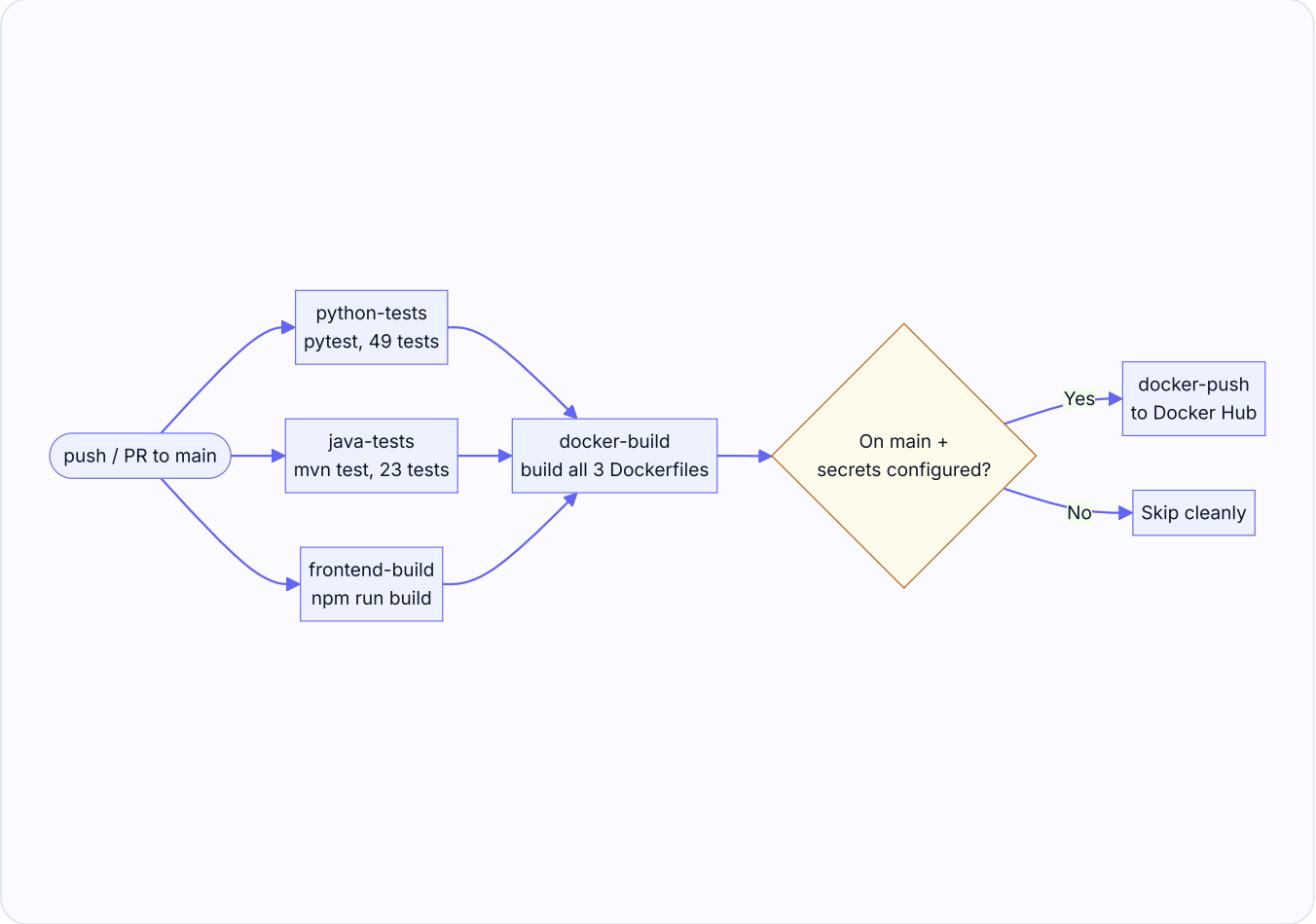

Fig 10.1 — Three independent jobs run in parallel; Docker builds only proceed if all three pass.

Job	What it verifies	Last run time
python-tests	49 pytest tests pass	~17s
java-tests	23 JUnit tests pass	~30s
frontend-build	Production Vite build compiles cleanly	~18s
docker-build	All 3 production Dockerfiles build successfully	~60s
docker-push	Optional — skips cleanly without Docker Hub secrets configured	~4s

VERIFIED, NOT ASSUMED

This pipeline was watched through to a real green completion on GitHub's hosted runners (`gh run watch`), not just locally syntax-checked.

11 Deployment Architecture

One-command full stack via Docker Compose

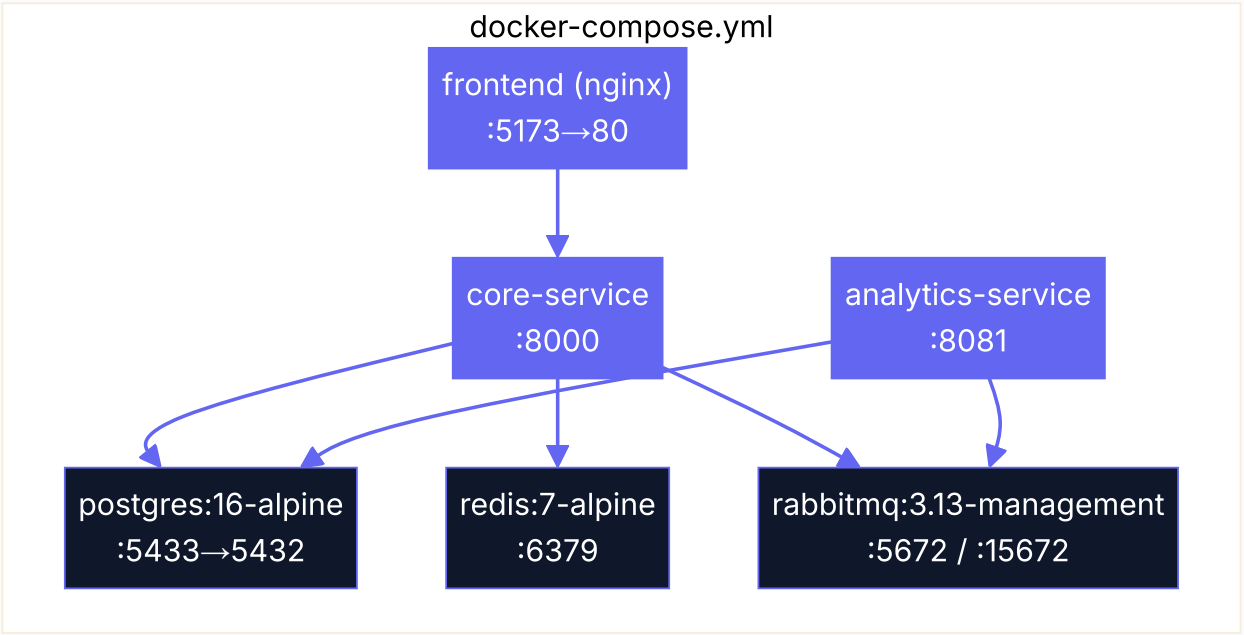

Fig 11.1 — All 6 services, health-check-gated startup ordering via depends_on conditions.

Quick start

```
git clone https://github.com/mahetab1357/url-shortener.git
cd url-shortener
cp .env.example .env
docker compose up -d
```

A separate `docker-compose.dev.yml` override swaps each Dockerfile's build target to `dev` and bind-mounts source code for hot reload (`uvicorn --reload` for Python, `mvn spring-boot:run` + Spring Boot DevTools for Java, real `vite dev` with HMR for the frontend).

12 Folder Structure

Repository layout

```
url-shortener/
├── core-service/          # Python/FastAPI
│   ├── app/
│   │   ├── routers/       # /shorten, /{short_code}, /api/urls/.../stats
│   │   ├── services/      # short-code gen, caching, click events, analytics client
│   │   ├── models/         # SQLAlchemy ORM models
│   │   ├── schemas/         # Pydantic request/response models
│   │   ├── db/                # engine/session, explicit schema init
│   │   └── mq/                  # RabbitMQ publisher
│   └── tests/              # 49 tests
├── analytics-service/      # Java/Spring Boot
│   └── src/main/java/.../analytics/
│       ├── controller/      # REST endpoint
│       ├── service/           # aggregation, parsers, RabbitMQ listener
│       ├── repository/         # Spring Data JPA
│       └── model/                # JPA entities — 23 tests
├── frontend/               # React/Vite
│   └── src/
│       ├── pages/             # ShortenPage, DashboardPage
│       ├── components/          # charts, link cards, copy button
│       ├── hooks/                # polling, localStorage-backed link list
│       └── api/                    # backend HTTP client
├── load-tests/              # Locust load test + recorded results
├── .github/workflows/         # CI pipeline
├── docker-compose.yml          # production-target stack
└── docker-compose.dev.yml       # hot-reload overrides
```

13 Future Improvements

Known gaps, stated honestly rather than hidden

Improvement	Why it's not done yet
Alembic / Flyway migrations	`create_all` / `ddl-auto: update` are dev-appropriate shortcuts, not production schema management
Real user accounts	The dashboard tracks "your" links via browser `localStorage` since no auth system exists; accounts would move this server-side
Geo-IP lookup	Raw client IP is captured but not resolved synchronously, to avoid adding external-API latency to every redirect
Rate limiting on /shorten	Not needed at current scale; would prevent abuse at higher scale
Custom domains	Out of scope for a demo project
Link expiry cron job	Expired links already 410 correctly; rows just aren't purged
Negative caching for 404s	Would absorb scan/enumeration traffic without hitting Postgres each time
Dead-letter queue	Failed click events are currently logged and dropped rather than retried elsewhere
Horizontally scaling analytics-service	Current aggregation logic relies on a single sequential consumer thread for correctness; multiple replicas would need atomic upserts or row locking
Maintained UA-parsing library	Hand-rolled parser covers common cases; `uap-java` would cover more real-world UA strings

14 Links & License

Where to find everything

Repository

github.com/mahetab1357/url-shortener

Key files in the repository

File	Contents
`README.md`	Full setup instructions, API docs, ER diagram
`load-tests/RESULTS.md`	Complete load test writeup, including the concurrency bug story
`.github/workflows/ci.yml`	CI pipeline definition
`docker-compose.yml` / `docker-compose.dev.yml`	Production and hot-reload stack definitions

License

MIT License — see `LICENSE` in the repository root.

THIS DOCUMENT

Generated as a static PDF export of the project's architecture, design rationale, and verified results. For the live, continuously-updated version of this information, see the repository's `README.md`.